

sdf-xarray: Interactive data reading and visualisation for EPOCH particle-in-cell simulations

Joel Luca Adams^{1*}, Peter Hill^{1*}, Liam Pattinson¹, Shaun Doherty¹, Sviatoslav Shekhanov¹, Chris Herdman¹, and Lawrence Dior¹

¹ York Plasma Institute, University of York, United Kingdom  Corresponding author * These authors contributed equally.

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) 
- [Repository](#) 
- [Archive](#) 

Editor: 

Submitted: 26 June 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](https://creativecommons.org/licenses/by/4.0/)).

Summary

EPOCH is a plasma physics code that outputs files using a custom binary format called SDF. sdf-xarray is a Python package which reads these files and parses them into structured N-D labelled datasets provided by Xarray. sdf-xarray has built-in functions for plotting, animation and unit conversion, allowing for simple interaction and manipulation for easy data analysis.

Statement of need

EPOCH (Arber et al., 2015), developed at the University of Warwick, is a Fortran-based Particle-In-Cell (PIC) code widely used in laser-plasma physics, primarily within the United Kingdom. The code was first developed at the University of Warwick in early 2008, prior to the standardisation of many modern output file formats and as a result uses the university's custom Self-Describing Format (SDF) binary files.

Integrating a modern output module like NetCDF (Rew et al., 1989) is challenging because the codebase accommodates one-, two-, and three-dimensional simulations across three variants: a regular grid geometry, a cylindrical geometry and a hybrid approximation. Multiple copies of the variants exist and are stored in forks and branches containing diverging commit histories making development of a unified module challenging.

State of field

Several SDF wrappers (Bennett, 2014) for use in visualisation software have been created, including but not limited to: VisIt (Childs et al., 2012); Matlab (Inc., 2022) which is closed-source and requires a paid license; OpenPMD (Huebl et al., 2015) which is not currently widely adopted; and a Python package called sdf_helper.

Python and Matplotlib (Hunter, 2007) are standard tools for plasma physics simulation analysis. The sdf_helper Python package supports this workflow by providing simple data extraction to NumPy (Harris et al., 2020) and custom Matplotlib plotting routines. Unfortunately, it lacks native capabilities for concatenating multiple SDF files into time-resolved datasets.

Developed as a modern successor to sdf_helper, sdf-xarray converts SDF files to Xarray (Hoyer & Hamman, 2017) datasets, enabling users to leverage core ecosystem features, including:

- Lazy data loading via Dask (Rocklin, 2015):** Instantiates pointers rather than loading the entire dataset into memory, substantially mitigating RAM constraints when handling

- large-scale files on the order of 10-100 GB.
- **Deferred computation:** Leverages Dask to postpone execution on data arrays until explicitly required or evaluated.
 - **Ecosystem interoperability:** Simplifies conversion of dataset arrays to NumPy or Pandas (McKinney, 2010) structures for downstream analysis.
 - **Interactive development:** Provides native compatibility with Jupyter notebooks (Granger & Pérez, 2021) to support iterative workflows.
 - **Built-in visualisation:** Contains integrated Matplotlib routines for rapid, native plotting of multidimensional data.
 - **Multi-file aggregation:** Facilitates the simultaneous opening and unified handling of split-file datasets.

In addition to leveraging Xarray's core ecosystem, sdf-xarray has comprehensive documentation which is actively maintained and makes use of Jupyter notebooks to illustrate the interactive nature of the package.

Both sdf_helper and sdf-xarray serve as wrappers around the C-based SDF-C library (Bennett & Wood, 2014). Although the underlying library historically required manual compilation via a Makefile, sdf-xarray utilises the modern Scikit-build-core (Schreiner et al., 2024) backend to automate this process for local builds. Furthermore, during the release cycle, sdf-xarray leverages cibuildwheel (cibuildwheel Developers, 2020) to distribute pre-compiled binaries across Windows, MacOS, and various Linux distributions. Originally, sdf_helper, released back in 2014, did not have this automated build process, making installation less consistent across operating systems. It also did not respect isolated Python virtual environments, causing some users to face system-wide dependency conflicts. Several months after the release of sdf-xarray in 2024, sdf_helper was updated to adopt a similar automated pipeline and published on PyPI as sdfx to alleviate these deployment challenges.

Software design

This package's design can be separated into three key sections: loading, plotting and rescaling.

Loading SDF files

Data ingestion follows a three-stage pipeline: raw binary reading via the underlying SDF-C library, decoding into Python dataclasses using Cython (Behnel et al., 2011), and parsing into a custom Xarray backend. During this process, the backend sanitises variable names to Pythonic snake_case, attaches simulation input data using epydeck (Hill & Adams, 2024), and omits heavy particle data by default to conserve memory.

For multi-file datasets, sdf-xarray aggregates data along a time dimension derived from each file's time attribute. However, this multi-file ingestion highlights a core limitation: Xarray strictly requires fixed grids, whereas EPOCH can output variable-grid data that changes with each step. Because these time-varying grids cannot natively coexist in a standard Xarray structure, sdf-xarray introduces a separate_times flag. This creates distinct time dimensions for variables with differing output frequencies, ensuring structural compatibility at the expense of a brief pre-evaluation RAM overhead.

Rescaling datasets

EPOCH outputs all data in base International System (SI) units. It can be convenient to convert these units when handling coordinate data; to that end a custom function was developed and an example can be found below:

```
# Convert the time to femtoseconds
ds = ds.epoch.rescale_coords(1e15, "fs", "time")
```

```
# Convert the x and y coords to microns
ds = ds.epoch.rescale_coords(1e6, "μm", ["X_Grid_mid", "Y_Grid_mid"])
```

82 This is often used in conjunction with generating plots and animations.

83 Converting variables can also be done by either directly manipulating the underlying NumPy array
84 or using the Pint (Pint Developers, 2012) package along with the Xarray API pint-xarray
85 (Magin et al., 2020).

86 Plotting

87 Xarray provides a simple interface for plotting variables using Matplotlib. An example of
88 which is:

```
import sdf_xarray as sdfxr

# Opens a single-file dataset
ds = sdfxr.open_dataset("0010.sdf")
ds = ds.epoch.rescale_coords(1e6, "μm", ["X_Grid_mid", "Y_Grid_mid"])

ds["Electric_Field_Ey"].epoch.plot()
```

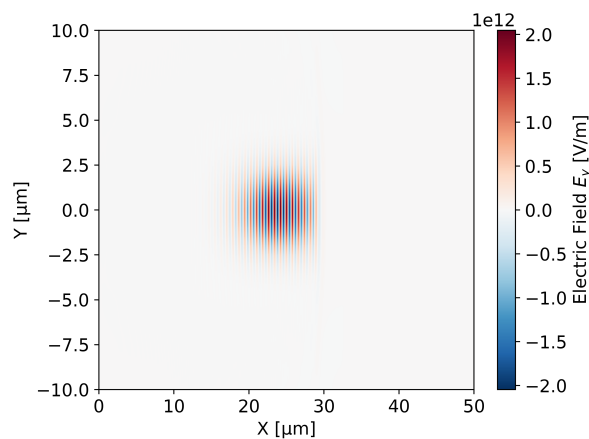


Figure 1: Plot of the electric field of a laser focusing in a vacuum, generated through sdf-xarray.

89 Animations

90 Building animations provides insight into how systems evolve over time. While Matplotlib
91 allows users to build animations, it can require quite a complicated setup in order to work with
92 Xarray. As a result, sdf-xarray contains a custom function that builds on top of Matplotlib's
93 implementation and provides a simple user interface. An example animation of a dataset
94 variable evolving over time is as follows:

```
import sdf_xarray as sdfxr

# Opens a multi-file dataset using globs or a list of paths
ds = sdfxr.open_mfdataset("*.sdf")
ds = ds.epoch.rescale_coords(1e15, "fs", "time")
ds = ds.epoch.rescale_coords(1e6, "μm", ["X_Grid_mid", "Y_Grid_mid"])

anim = ds["Electric_Field_Ey"].epoch.animate()
```

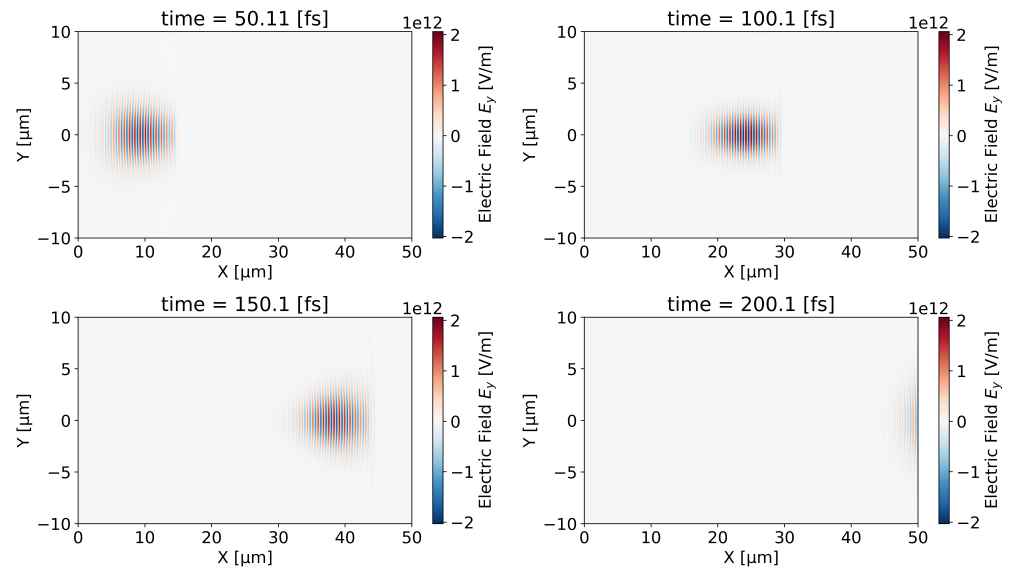


Figure 2: Frames from an animation of the electric field of a laser focusing in a vacuum, generated through sdf-xarray.

95 The minimum and maximum values of a variable can change between each SDF file, so when
96 generating an animation the limits must be found over the whole dataset. This behaviour
97 exists by default in sdf-xarray animations.

98 EPOCH provides a 'moving window' feature which shifts the axes as a simulation
99 progresses (e.g. to track a laser pulse propagating through a long block of
100 plasma). This saves computational resources by simulating only the most dynamic
101 region of the domain rather than its full extent. sdf-xarray animations allow the
102 user to specify a boolean value for this `moving_window` functionality to follow the
103 simulation box instead of maintaining fixed axes.

104 On top of building single variable animations, users can utilise the `animate_multiple()`
105 function to overlay multiple variables.

106 Further examples and explanation of animations can be seen in the [documentation](#).

107 Research impact statement

108 This library was originally developed for the machine learning pipeline toolkit [BEAM](#). The initial
109 software architecture was designed by Peter Hill, with subsequent maintenance, optimization,
110 and feature iteration led by Joel Adams and community contributors. Since its inception, the
111 library has been widely adopted by EPOCH users at the York Plasma Institute, fostering a
112 collaborative development environment where active researchers contribute new features to
113 the codebase.

114 Beyond its foundational deployment at the University of York, sdf-xarray has achieved broader
115 institutional adoption, seeing active use by researchers at the University of Strathclyde, Queen's
116 University Belfast and First Light Fusion. In addition to driving primary research, the library
117 serves a distinct educational role; it has been integrated into a course on PIC codes delivered
118 to undergraduate students at the University of York and served as the primary data-interfacing
119 utility for both the 2025 and 2026 international EPOCH user workshops.

120 By adhering strictly to FAIR (Findable, Accessible, Interoperable, and Reusable) data principles,
121 sdf-xarray alongside its sister BEAM packages, epyscan and epydeck ([Hill et al., 2024](#))

maintains a transparent, community-driven development model that encourages external contributions. In recognition of this commitment to open science and its measurable impact on the plasma physics community, the BEAM project was awarded the 2025 University of York Open Research Award in the Postgraduate Researcher (Sciences) category.

AI usage disclosure

Some of the code and documentation in this library was partially developed with assistance from the GPT models by Google's Gemini, Anthropic's Claude and OpenAI's ChatGPT. The majority of the codebase was developed by humans and all generated code was verified by several developers prior to release.

Acknowledgements

This project's initial development was funded by the PlasmaFAIR project, EPSRC Grant EP/V051822/1

We acknowledge the University of Warwick EPOCH development team for their support of the project and indirect contributions to this project by several anonymous contributors.

References

- Arber, T. D., Bennett, K., Brady, C. S., Lawrence-Douglas, A., Ramsay, M. G., Sircombe, N. J., Gillies, P., Evans, R. G., Schmitz, H., Bell, A. R., & Ridgers, C. P. (2015). Contemporary particle-in-cell approach to laser-plasma modelling. *Plasma Physics and Controlled Fusion*, 57(11), 113001. <https://doi.org/10.1088/0741-3335/57/11/113001>
- Behnel, S., Bradshaw, R., Citro, C., Dalcin, L., Seljebotn, D. S., & Smith, K. (2011). Cython: The best of both worlds. *Computing in Science Engineering*, 13(2), 31–39. <https://doi.org/10.1109/MCSE.2010.118>
- Bennett, keith. (2014). SDF. In *GitHub repository*. Warwick Plasma Group. <https://github.com/epochpic/SDF>
- Bennett, keith, & Wood, A. (2014). SDF_C. In *GitHub repository*. Warwick Plasma Group. https://github.com/epochpic/SDF_C
- Childs, H., Brugger, E., Whitlock, B., Meredith, J., Ahern, S., Pugmire, D., Biagas, K., Miller, M. C., Harrison, C., Weber, G. H., Krishnan, H., Fogal, T., Sanderson, A., Garth, C., Bethel, E. W., Camp, D., Rubel, O., Durant, M., Favre, J. M., & Navratil, P. (2012). *High Performance Visualization—Enabling Extreme-Scale Scientific Insight*. <https://doi.org/10.1201/b12985>
- cibuildwheel Developers. (2020). cibuildwheel. In *GitHub repository*. Python Packaging Authority. <https://github.com/pypa/cibuildwheel>
- Granger, B. E., & Pérez, F. (2021). Jupyter: Thinking and storytelling with code and data. *Computing in Science & Engineering*, 23(2), 7–14. <https://doi.org/10.1109/MCSE.2021.3059263>
- Harris, C. R., Millman, K. J., Walt, S. J. van der, Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., Kerkwijk, M. H. van, Brett, M., Haldane, A., Río, J. F. del, Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>

- 163 Hill, P., & Adams, J. L. (2024). *epydeck: EPOCH input file reader/writer*. <https://doi.org/10.5281/zenodo.15586339>
- 164
- 165 Hill, P., Adams, J. L., & Dior, L. (2024). *epyscan: Generate EPOCH campaigns over a*
- 166 *parameter space using different sampling methods*. <https://doi.org/10.5281/zenodo.17106949>
- 167
- 168 Hoyer, S., & Hamman, J. (2017). Xarray: N-D labeled arrays and datasets in Python. *Journal*
- 169 *of Open Research Software*, 5(1). <https://doi.org/10.5334/jors.148>
- 170 Huebl, A., Lehe, R., Vay, J.-L., Grote, D. P., Sbalzarini, I., Kuschel, S., Sagan, D., Pérez,
- 171 F., Koller, F., & Bussmann, M. (2015). *openPMD: A meta data standard for particle*
- 172 *and mesh based data*. <https://github.com/openPMD>; Zenodo. <https://doi.org/10.5281/zenodo.591699>
- 173
- 174 Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science &*
- 175 *Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>
- 176 Inc., T. M. (2022). *MATLAB version: 9.13.0 (R2022b)*. The MathWorks Inc. <https://www.mathworks.com>
- 177
- 178 Magin, J., Nicholas, T., Pflüger, M., Thielen, J., Keys, S., Patel, B., Marel, M. van der,
- 179 & Bovy, B. (2020). *pint-xarray*. In *GitHub repository*. Xarray Contributions. <https://github.com/xarray-contrib/pint-xarray>
- 180
- 181 McKinney, Wes. (2010). Data Structures for Statistical Computing in Python. In Stéfan van
- 182 der Walt & Jarrod Millman (Eds.), *Proceedings of the 9th Python in Science Conference*
- 183 (pp. 56–61). <https://doi.org/10.25080/Majora-92bf1922-00a>
- 184 Pint Developers. (2012). *Pint*. In *GitHub repository*. Hernan Grecco. <https://github.com/hgrecco/pint>
- 185
- 186 Rew, R., Davis, G., Emmerson, S., Cormack, C., Caron, J., Pincus, R., Hartnett, E., Heimbigner,
- 187 D., Appel, L., & Fisher, W. (1989). *Unidata NetCDF*. NSF Unidata. <https://doi.org/10.5065/D6H70CW6>
- 188
- 189 Rocklin, M. (2015). Dask: Parallel computation with blocked algorithms and task scheduling.
- 190 In K. Huff & J. Bergstra (Eds.), *Proceedings of the 14th python in science conference* (pp.
- 191 130–136). <https://www.dask.org>
- 192 Schreiner, H., III, Fillion-Robin, J.-C., & McCormick, M. (2024, July). *Scikit-build-core*.
- 193 <https://doi.org/10.25080/FMKR8387>